12 二元搜尋樹 解答

12-E01 中序走訪與 BST 判定

推導

左子樹中序為 3、6，根為 5，右子樹中序為 7、8，完整中序是 3、6、5、7、8。

結論與檢核

序列不是非下降，且 6 位於根 5 的左子樹卻大於 5，所以不是 BST。只看父子 3<5、6>3 會漏掉跨層違規。

12-E02 successor 走法

推導

中序順序由結構決定。x 無右子時，從 x 往父移動，直到首次由左子樹進入某祖先。由 4 上到 2：x 是 2 的右子，繼續；再到 6：前一個 2 是 6 的左子，停止。

結論與檢核

故 successor 是 6。停止條件是 child 等於 parent.left，不是 key 的數值比較；若到根從未由左子進入則無 successor。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12-E03 雙子刪除

推導

若 12 不是 10 的直接右子，先以 13 替代 12 的舊位，令 13.parent 指向 12 的舊父，再把原 10.right 接為 12.right 並更新 parent。之後以 12 替代 10，令 12.left=10.left 並更新 parent。若 12 本來就是 10.right，保留它原本的右子 13，跳過前面的移植；不可把 13 再接兩次。

結論與檢核

根與相關節點的 parent 都需同步更新。完成後中序次序保持，10 已移除，13 沒有遺失。

12-E04 後繼Right child的移植

推導

後繼y25、舊父30、右子27。先30.left=27、27.parent=30；再25.right=40、40.parent=25；root=25、25.parent=NIL；最後25.left=10、10.parent=25。其餘邊不改，得到中序5,10,15,25,27,30,35,40,50。

12 二元搜尋樹 解答 （續）

結論與檢核

原node20的handle失效，node25的handle仍指同一物件但位置改變；若只把25的key/data複製進node20再刪node25，失效的會是另一個identity，不能視為同一API行為。

12-E05 排列均勻不等於Shape均勻

推導

213、231同為root2孩子1/3，height1；123、132、312、321各產生不同的三節點鏈，height2。均勻排列的期望(2×1+4×2)/6=5/3；五shape均勻則(1+4×2)/5=9/5。

結論與檢核

模型不同就有不同分母與機率，不能把所有樹形等可能代入隨機插入的遞迴。正確模型root秩均勻，條件於root兩側相對排列也均勻。

12-E06 平均Depth不控制最大Depth

推導

原N個node總depth為Theta(N log N)。新鏈的depth依序h,h+1,⋯,h+L−1，總Lh+L(L−1)/2=O(sqrt(N)log N+N)，所以全部總depth仍Theta(N log N)。n=Theta(N)，平均depth=Theta(log n)。

結論與檢核

但height=h+L−1=Theta(sqrt(n))，遠大於log n。平均node的depth小不排除少數很深的node；正文改直接控制2^{height}的期望，再用Jensen/Markov處理最大值。

12 二元搜尋樹 解答 （續）

原創驗算程式附錄: bst_checks.py

```
"""Validate real pointer operations, all small insertion orders, and exact height moments."""
from itertools import permutations
from fractions import Fraction
from math import comb
import random


class Node:
    def __init__(self,key):
        self.key=key
        self.left=self.right=self.parent=None


class Tree:
    def __init__(self,keys=()):
        self.root=None
        self.events=set()
        for key in keys:
            self.insert(key)
    def search(self,key):
        node=self.root
        while node is not None and key!=node.key:
            node=node.left if key<node.key else node.right
        return node
    def insert(self,key):
        parent=None
        node=self.root
        while node is not None:
            if key==node.key:
                return node
            parent=node
            node=node.left if key<node.key else node.right
        new=Node(key)
        new.parent=parent
        if parent is None:
            self.root=new
        elif key<parent.key:
            parent.left=new
        else:
            parent.right=new
        return new
    def transplant(self,u,v):
        parent=u.parent
        if parent is None:
            self.root=v
        elif u is parent.left:
            parent.left=v
        else:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12 二元搜尋樹 解答 （續）

```
            parent.right=v
        if v is not None:
            v.parent=parent
    def delete(self,z):
        if z.left is None:
            self.events.add('leaf' if z.right is None else 'only-right')
            self.transplant(z,z.right)
        elif z.right is None:
            self.events.add('only-left')
            self.transplant(z,z.left)
        else:
            y=z.right
            while y.left is not None:
                y=y.left
            if y.parent is not z:
                self.events.add('two-deep')
                if y.right is not None:
                    self.events.add('deep-successor-has-right')
                self.transplant(y,y.right)
                y.right=z.right
                y.right.parent=y
            else:
                self.events.add('two-direct')
            self.transplant(z,y)
            y.left=z.left
            y.left.parent=y


def neighbor(node,forward=True):
    child='right' if forward else 'left'
    other='left' if forward else 'right'
    steps=0
    if getattr(node,child) is not None:
        node=getattr(node,child)
        steps+=1
        while getattr(node,other) is not None:
            node=getattr(node,other)
            steps+=1
        return node,steps
    parent=node.parent
    steps+=1
    while parent is not None and node is getattr(parent,child):
        node,parent=parent,parent.parent
        steps+=1
    return parent,steps


def validate(tree,expected):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12 二元搜尋樹 解答 （續）

```
    seen={}
    def walk(node,parent,lo,hi):
        if node is None:
            return [],-1
        assert node not in seen.values()
        assert node.parent is parent and lo<node.key<hi
        seen[node.key]=node
        left,lh=walk(node.left,node,lo,node.key)
        right,rh=walk(node.right,node,node.key,hi)
        return left+[node.key]+right,1+max(lh,rh)
    ordered,height=walk(tree.root,None,float('-inf'),float('inf'))
    assert ordered==sorted(expected)
    for i,key in enumerate(ordered):
        node=seen[key]
        assert tree.search(key) is node
        successor,_=neighbor(node)
        predecessor,_=neighbor(node,False)
        assert successor is (seen[ordered[i+1]] if i+1<len(ordered) else None)
        assert predecessor is (seen[ordered[i-1]] if i else None)
        current=node
        total=0
        for k in range(1,len(ordered)-i+1):
            current,cost=neighbor(current)
            total+=cost
            assert total<=4*k+3*height+5
            if current is None:
                break
    return seen,height


def height(node):
    return -1 if node is None else 1+max(height(node.left),height(node.right))


def main():
    deletions=0
    events=set()
    for n in range(1,7):
        for order in permutations(range(n)):
            for key in range(n):
                tree=Tree(order)
                before,_=validate(tree,set(range(n)))
                tree.delete(before[key])
                after,_=validate(tree,set(range(n))-{key})
                assert all(after[k] is before[k] for k in after)
                events|=tree.events
                deletions+=1
    keys=[20,10,40,5,15,30,50,25,35,27]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12 二元搜尋樹 解答 （續）

```
    tree=Tree(keys)
    before,_=validate(tree,set(keys))
    assert neighbor(before[20])[0] is before[25]
    assert neighbor(before[27])[0] is before[30]
    tree.delete(before[20])
    after,_=validate(tree,set(keys)-{20})
    assert tree.root is before[25] and tree.root.parent is None
    assert after[30].left is before[27] and after[27].parent is before[30]
    assert after[25].right is before[40] and after[40].parent is before[25]
    events|=tree.events
    rng=random.Random(20261024)
    tree=Tree()
    reference=set()
    for _ in range(2000):
        key=rng.randrange(70)
        if rng.randrange(2):
            tree.insert(key)
            reference.add(key)
        elif key in reference:
            tree.delete(tree.search(key))
            reference.remove(key)
        validate(tree,reference)
        events|=tree.events
    for key in sorted(reference):
        tree.delete(tree.search(key))
    validate(tree,set())
    assert events=={'leaf','only-right','only-left','two-deep','two-direct','deep-successor-
has-right'}
    moments=[]
    permutation_count=0
    for n in range(1,9):
        heights=[height(Tree(order).root) for order in permutations(range(n))]
        expected_height=Fraction(sum(heights),len(heights))
        expected_exponential=Fraction(sum(2**h for h in heights),len(heights))
        assert expected_exponential<=Fraction(comb(n+3,3),2)
        permutation_count+=len(heights)
        moments.append((n,str(expected_height),str(expected_exponential)))
    assert moments[2][1]=='5/3'
    for n in range(1,80):
        b=[Fraction(comb(i+3,3),2) for i in range(n+1)]
        assert Fraction(4,n)*sum(b[:-1])==b[-1]
    print(f"PASS BST_single_deletions={deletions} mixed_operations=2000
all_parent_global_bounds_identity_and_neighbors=True events={sorted(events)}
height_permutations={permutation_count} exact_moments={moments}")


if __name__=='__main__':
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12 二元搜尋樹 解答 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study